QTF

Auth Person

Biometric → Fuzzy Extraction → Gen Dilithi pri key at Terminal → Term. comp Priv key ag public key

↓

Terminal tells the server p is auth

PQC – Dili

Auth Termi

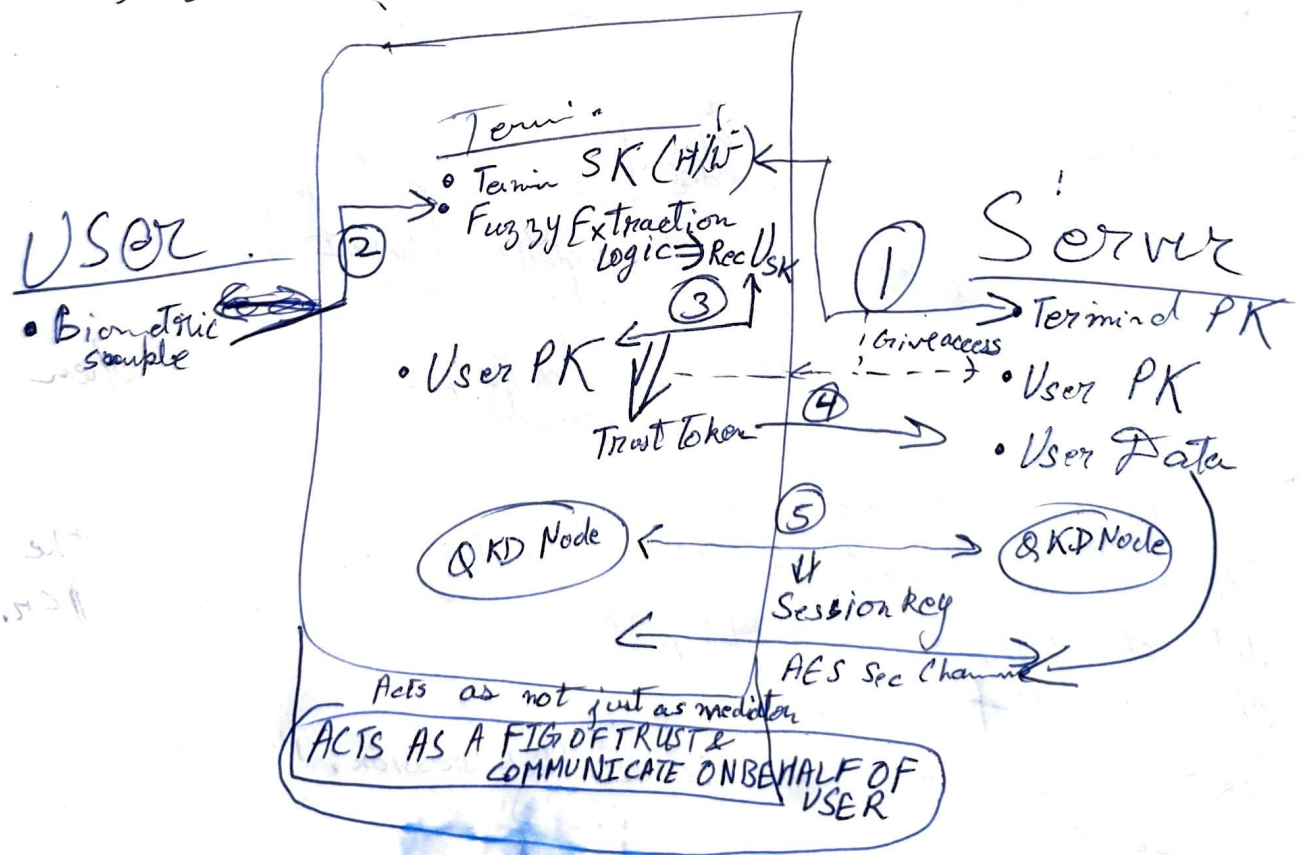
Hardware backed Dil Private Key at terminal

← Signature (Term verified) Term gets access to User PK DB

Rest Pub Key at server

Sec Comm on Termi ↔ Serve

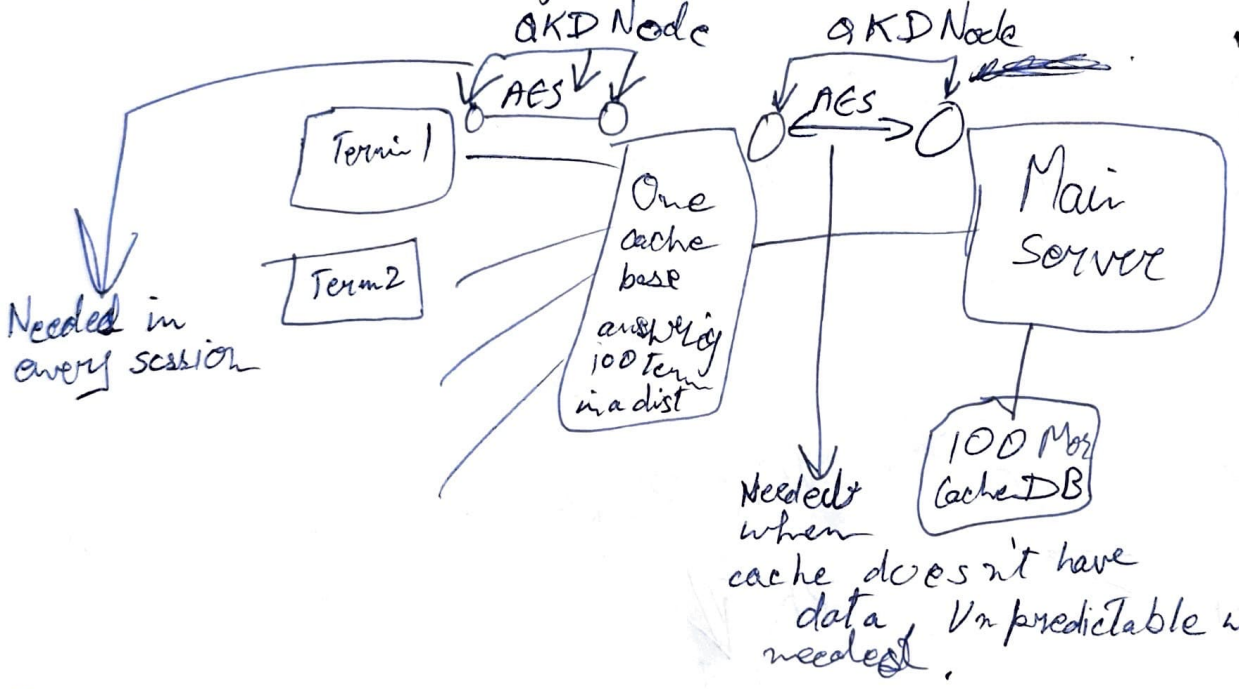
- QKD b/w server & Terminal



Question on this arch?

(1) How do we make scalable?
BB84 / any QKD dies at 100km. 1% Photon survive just fibre loss. Add noise due to quantum operators. System dies before ever establishing.
Keeping decentral server at every 100 km. 100% NP.

(2) Server is central authority?
- Basic pricible of Crypto. Single central serv can't, CAN'T, sustain the data load we expect,
- Data caching is possible. But we have to keep the cached data secure, Arch changes slightly



(3) Dilithium keys are hyge.
Di keys are almost 3X the RSA. How do we manage the data of 140,00,00,000 ppl, 10,00,000 Termial. 1.3KB X 14 Cr. of just keys + data.

(4) QKD takes hyge Time to establish session. How do we optimise it?
Possible sol: Keeping keys ready for AES session in a key bank. But Then another 100% Resp of keeping the key bank sec.

(5)

Dilithin key resolution.

The paper I saw have proved Dilithin to be a bigger ~~to~~ than QKD itself. QKD can be resolved by session key banking to some extent. Dilithin key resolving have to by dynamic for each user. How do we manage that?

(6)

Sir's arch

1) Sending bio sample via QSDC. ✗
→ Arch is based on no template
2) Send derived key/signature
→ Key is huge & Sign is huge
→ Sign can't be altered. Guaranteed by PQC lattice Math
3) Send ing Trust token ✗
→ ~~Not~~ Trust token gets scc. But what about further things?
4) Sending data ✗
→ Unfeasible 1GB data ⇒ 8.6 Billion bit ⇒ 4.3 billion Bell Pair
5) Sending AES key?
~~Why no~~ Wouldn't it be same as QKD. Idiot.
Wait. Not entirely impossible. Treat key as message. send key from Term ↔ serv. Serv recv. key. Now do AES (?)

↳ How much better QSDC is compared to QKD?

- → Efficiency
- → Security
- → Reliability
- → Physical resrc req
- → Noise Resistance

Quantum Trust Flow Refined

1. User starts session
   - ↳ Terminal gathers metadata
2. Terminal auth
   - → Terminal has H/w sec priv key. (T_{SK})
   - → Gathered MD + status + Dil sig ⟶ Server
   - ↳ Server verifies sign
3. Server auth
   - → Server MD + S_{PK} Dil. Sig ⟶ Terminal
   - ↳ Terminal Verifies
4. User gives ID credential (email/Id)
   - ↳ Terminal : UUID ⟶ Server
     Server : U_{PK} + Recreation helper data ⟶ Terminal
5. Iris scan &
   - → Term extracts bio data locally
   - → Fuzzy extract + helper data → U_{SK} Dil reconstructed
   - → Sign Dil chall. with U_{SK} → Verify against U_{PK}
     - Not verified → Session Terminates
     - Verified
       - → QAuth Token {MD, status, bio conf, ATS, ETS, req serv}
       - ↳ Terminal Dil. signs QAuth token

(6) Server validates & Auth Token

- → Verifies Message authenticity
- → Checks ETS
- → Server initiates & KD sesh with terminal
- → Sec key material shared (Fresh every time)

(7) Key gen & AES sesh

- → Key material KCV : $F \rightarrow (K_{enc}, K_{auth}, K_{CAuth}, K_{randu}$
- → Keys used to stat AES session

(8) Cont auth

- → Session is a renewable lease
- → Once initial auth accepts & AES starts:
  - → Detect face / eye presence
  - → Proximity
  - → Graze din
  - → CV
- → Full Iris scan
- → Conf < Threshold
- → Inactivity

(9) On-basis of (8), cont gen CAuth tokens verified by server using MD.